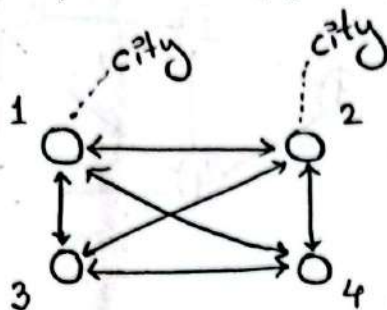


TRAVELING Salesman problem (TSP)

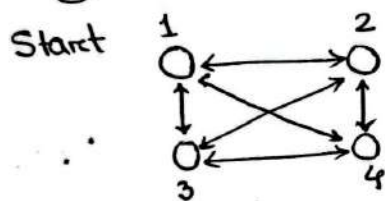
; np complete problem ;
(Hamilton Algo)



	1	2	3	4
1	0	10	15	20
2	5	0	25	10
3	15	30	0	5
4	15	10	20	0

☐ Salesman wants to promote his product in market of different cities.

He will (suppose starts from 1) start from one end and again will reach same end (ie. 1).



greedy will say, if you start from 1 node, you will select 0, 10, 15, 20. from 1 to 2

if you move from 1 node to 2 node, then again u move from 2 node to 1 node; you will be required time.

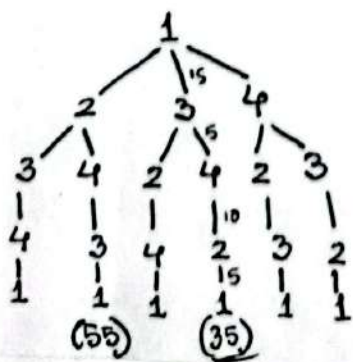
1
↓ 10
2
↓ 10
4
↓ 20
3
↓ 15
1

[Time will be required more, if you repeat same city]

55 weight/cost. ; Greedy Programming.

* According to Brute Force, pass all possibility.

; Dynamic Programming.

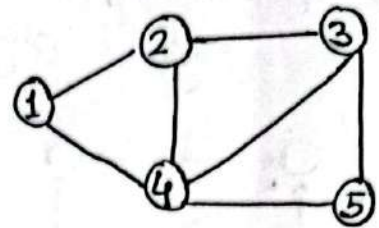


1 ... n → 1

exponential
Time-Complexity.

1 2 3 4 | 1
1 2 4 3 | 1
1 4 3 2 | 1
n-1
n-1
O(n!)
O(n^n)

Representation of Graph,



① Adjacency Matrix,

② Adjacency List,

Adjacency Matrix,

It is a matrix $A[n][n]$,
where n is no. of vertices,

$$\begin{cases} a[i][j] = 1, & \text{if } i \& j \text{ are adjacent} \\ = 0, & \text{otherwise} \end{cases}$$

$m \times n$
no. of Rows. no. of Columns.

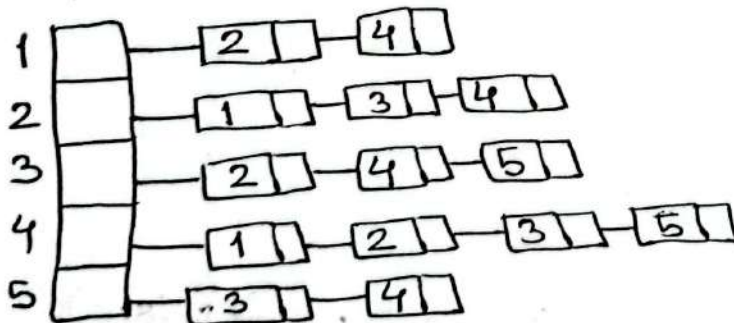
space complexity $\theta(n^2)$

$i \downarrow j \rightarrow$	1	2	3	4	5
1	0	1	0	1	0
2	1	0	1	1	0
3	0	1	0	1	1
4	1	1	1	0	1
5	0	0	1	1	0

5x5

For each vertex, 1 linked list - present.

↳ Better use
↳ Dense Graph
(no loop, so diagonal zero)



Space Complexity: $\theta(n+2e)$

1/2 times written

↳ Better use

↳ Sparse Graph,

(few vertices - few connections)

String Matching

↳ necessary in many applications

Detecting
✓ Plagiarism

✓ web searches

✓ DNA matching

✓ Network Intrusion Detection Systems

Other Algorithms -

□ Naive Algo - method:

✓ check each place string could be 1 by 1.

✓ No pre-processing.

✓ Matching: $O(nm)$

□ Knuth-Morris-Pratt:

✓ Uses prefix table to avoid re-examining previously matched substrings.

✓ Pre-processing: $O(m)$

✓ Matching: $O(n)$.

28/9/24

③ Why Boyer-Moore-Horspool Algorithm?
Efficiency on Average.

↳ often achieves $O(n)$,

faster than brute force.

□ speed, simplicity & practicality.

□ text processing.

Problem,

✓ String Matching (Matching a pattern with a
✓ Given pattern $P(1 \dots n)$ and text $T(1 \dots m)$
defined over alphabet Σ , find one or all
occurrences of P in T .

Naïve Algorithm;

Start with pattern/string; 'ANT'
Find in text: 'I AM ANTELOPE'
(leftmost) ✓

I A M A N T E L O P E
↑
* A N T

I A M A N T E L O P E
↑ ↑
A N T

I A M A N T E L O P E
↑ ↑ ↑
A N T
✓

* Worst case,

a a a ... b input text (length
a a a .. b pattern (length m)

* Mismatch $(n-m)$ times

* Success at $(n-m)+1$ iterations.

Total comparisons: $((n-m)+1) \times m$

$\Theta(nm)$ Quadratic time.

Boyer Moore Horspool Algo.
Primarily, make 'Bad match table'.
(rightmost) ✓

Pattern: 'tooth' text: 'trusthardtoothbrushes'

Construct Bad Match table:
value = length - index - 1 (Even)

TOOTH
0 1 2 3 4

Length = 5

Letter
value

T

O

H

*

1st d: $T = 5 - 0 - 1 = 4$
 $O = 5 - 1 - 1 = 3$

$O = 5 - 2 - 1 = 2$
 $T = 5 - 3 - 1 = 1$

4
4
4
4

3
2
2

previous ✓
5

letter	T	O	H	*
value	1	2	5	5

TRUSTHARDTOOTHBRUSHES
 T O O T H A R D T O O T H B R U S H E S
 ↑ look
 ↑ shift one space
 x

TRUSTHARDTOOTH...
 T O O T H A R D T O O T H ...
 ↑ ↑ ↑ shift five spaces,
 x ✓ ✓

TRUSTHARDTOOTHB...
 T O O T H A R D T O O T H B ...
 x
 T O O T H shift 2x

TOOTH B
 T O O T H → 1 space
 x

T O O T H
 ↑ ↑ ↑ ↑ ↑
 T O O T H

Worst same as naïve, ✓

1^n input text (length n)

0111...1 pattern (length m)

Worst case, $O(nm)$

Best,

1^n text

0^m pattern

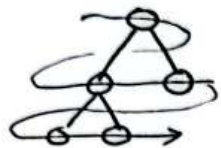
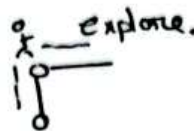
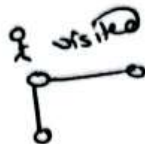
Best case, $O(m/n)$

avg: $O(\frac{m}{|T|})$

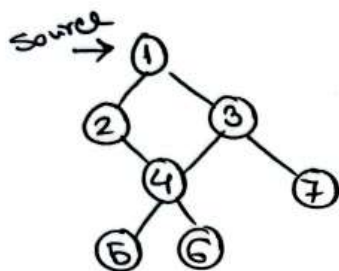
Graph TRAVERSAL (visit & explore)

Am

BFS



eg. taking food,



Queue (DS)

1 2 3 4 7 5 6

1 2 3 4 7 5 6

working (level by level)



Result / Output

① Preorder: ABDEC

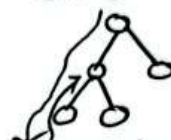
② Inorder: DBE A C

③ Postorder: DEBC A

"DFS"

"BFS": Level by Level.

DFS

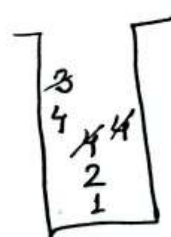


backtracking ✓

(When dead end comes)

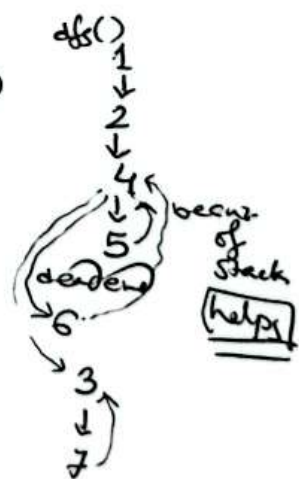
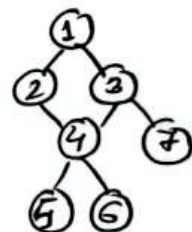
eg. taking life goals - subjects

DFS



Stack

LIFO (DS)



1 2 4 5 6 3 7

bfs & dfs $\rightarrow O(V+E)$

Application:

- social media
- network
- minimum spanning tree

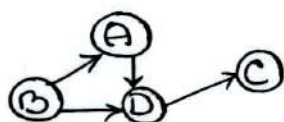
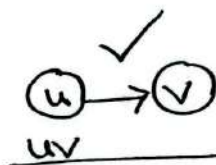
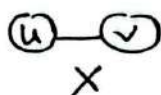
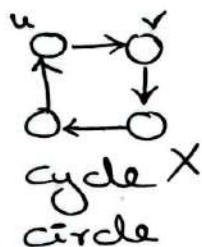
Topological Sort

It is linear ordering of graph vertices such that, for every directed edge uv from vertex u to vertex v , u comes before v in the ordering.

→ Applicable on DAG (Direct Acyclic Graph)

→ Linear Running Time Complexity, $O(V+E)$

→ Topological Sort is not Unique.



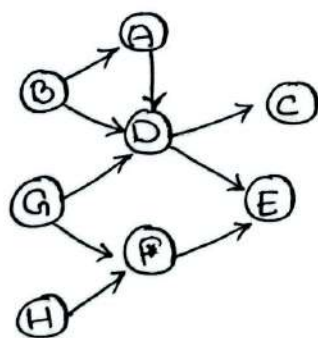
Topological Order: BADC ✓

ABCD X

BDCA X

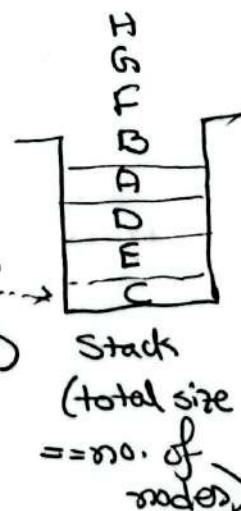
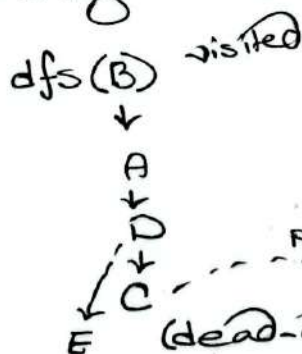
How we will write it?

(using DFS)



end of v plus E
 $O(V+E)$

dfs(G)
↓
F



stack $O(V)$

extra space complexity

H G F B A D E C

non-unique X

multiple result possible

Handwritten scribbles at the bottom left.

(BFS)

Kahn's Algorithm,

$L \leftarrow$ Empty list that will contain the sorted elements.

$S \leftarrow$ Set of all nodes with no incoming edge.

While S is not empty do
remove a node n from S
add n to L

for each node m with an edge e from n to
remove edge e from the graph

if m has no other incoming edges then
insert m into S

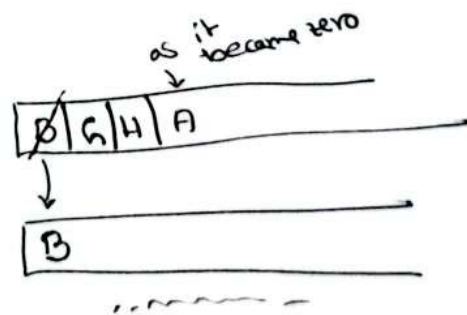
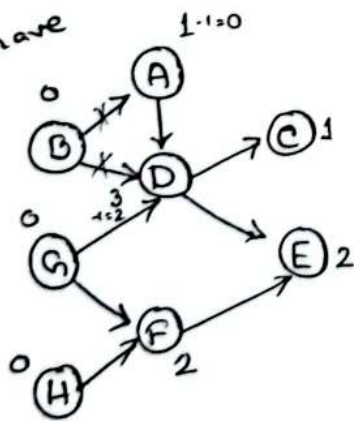
if graph has edges then

return error (graph has at least one cycle)

else

return L (a topologically sorted order)

* DAG
(will have
at least
one vertex
with zero
indeg)

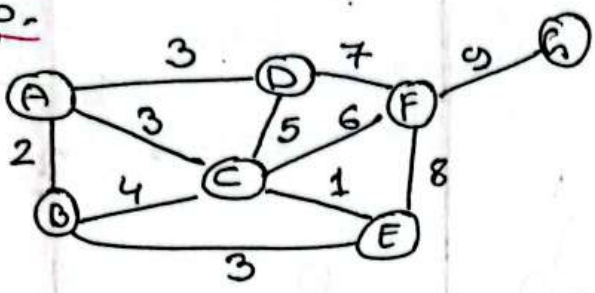


Step 1: Write indegree?
(incoming)

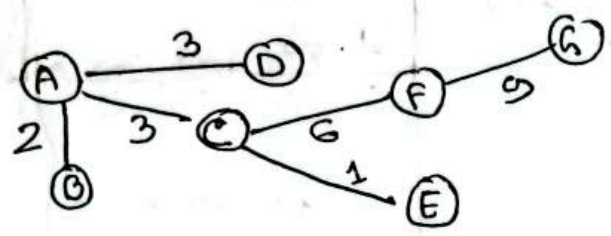
Step 2: remove connected edge (-1)
after adding in
Queue.

Sam
1/11/23
Wednesday
8:54 am

prim's algo.



arbitrary Node



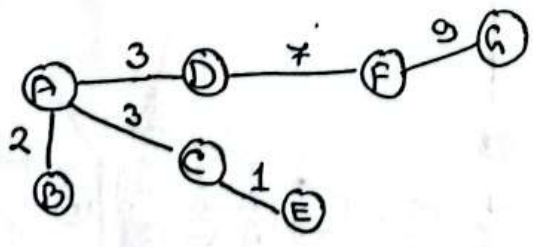
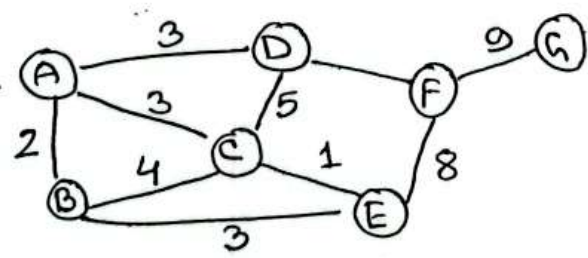
visited = {A, B, C, E, D, F, G}

Time Complexity -
adjacency matrix searching $\rightarrow O(V^2)$ edge = 24

Binary Heap and Adjacency List $\rightarrow O(V \log V + E \log V)$

ruskal's algo.
Kruskal
pronounce.

test taking lowest edge weight

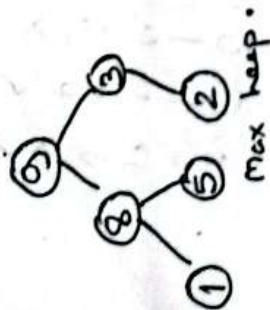


- C-E $\rightarrow$ 1
- A-B $\rightarrow$ 2
- A-D $\rightarrow$ 3
- A-C $\rightarrow$ 3
- B-E $\rightarrow$ 3 X
- D-C $\rightarrow$ 5 X
- D-F $\rightarrow$ 7 $\checkmark$
- F-G $\rightarrow$ 9 $\checkmark$

Time Complexity -
 $O(E \log E)$ ~ Merge Sort

Heap: ordered binary tree.
Max-heap: parent > child.

Time Complexity
 $O(n \log n)$



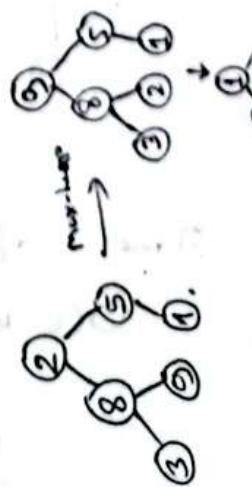
Max heap.

build-max-heap: creates max heap from unsorted array.

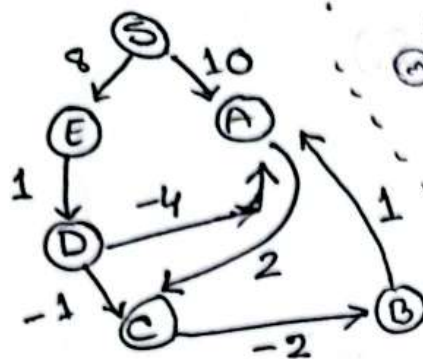
max Heapify: similar to build-max-heap, but assumes part of array is already sorted.



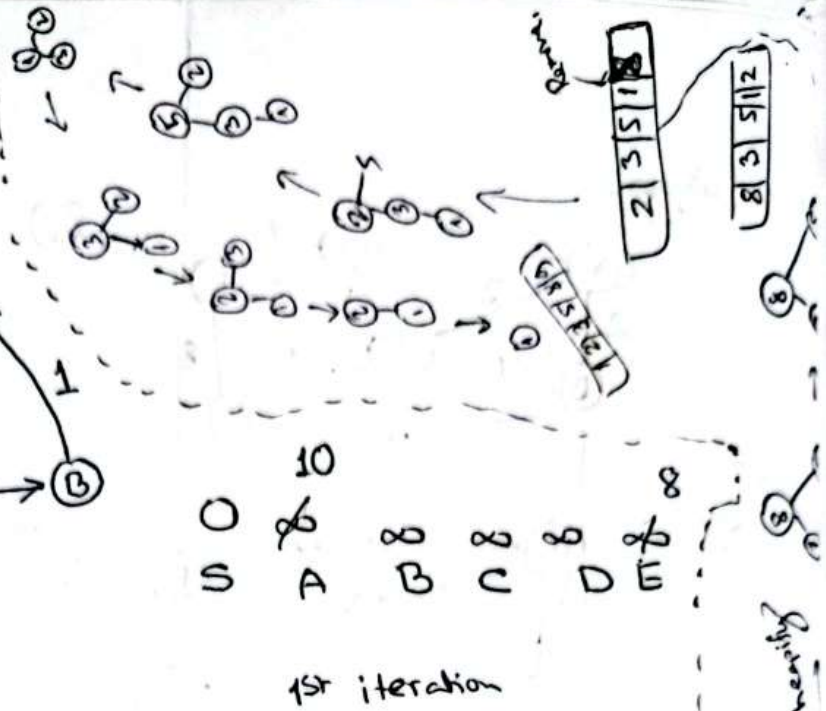
- ① Create Max Heap.
- ② Remove Largest Item.
- ③ Place item in sorted position.



Bellman-Ford algo.



Time Complexity,
 $O(|V| \cdot |E|)$



	10				
S	A	B	C	D	E

1st iteration

	10	$12-2=10$	$10-1=9$	$12-1=11$	8
S	A	B	C	D	E
	$9-4=5$	10	$9-1=8$	12	9
A		B	C	D	E

2nd iteration

	5	$8-1=7$	8	9	8
S	A	B	C	D	E

3rd iteration

	5	7	7	9	8
S	A	B	C	D	E

4th iteration



item/objects $\rightarrow$ 1 2 3 4 $\left\{ \begin{array}{l} \text{max weight} = 5 \\ n = 4 \end{array} \right.$
 weight $\rightarrow$ 2 3 4 2
 value/profits $\rightarrow$ 4 3 5 1

$i \downarrow w \rightarrow$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	4	4	4	4
2	0	0	4	4	4	4
3	0	0	4	4	5	
4	0					

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	4	4	4	4
2	0	0	4	4	7	7
3	0	0	4	4	7	7
4	0	0	4	4	7	9

$i \backslash w$	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	4	4	4	4
2	0	0	4	4	7	7
3	0	0	4	4	7	7
4	0	0	4	4	7	9

1 2 3 4
 3 2 5 4
 4 3 6 5

0 0 0 0 0 0
 0 0 0 1 1 1
 0 0 3 4 1 1
 0 1 0 3 4 1 1

$$B[i, w] = \max(B[i-1, w], B[i-1, w-w[i]] + v[i])$$

1 2 3 4
 3 2 5 4
 4 3 6 5
 0 0 0 0 0 0
 0 0 0 1 1 1
 0 0 3 4 1 1
 0 1 0 3 4 1 1

N-queens:



- It's a classic combinational problem, to place n queens on $n \times n$ chessboard, so that no two attack.

- Rules: No two of them should be on same row, column or diagonal.

- All solns to n -queens problem can therefore be represented as n -tuples. $(x_1, x_2, \dots, x_n)$, where x_i is column where Queen 'i' is placed, 2nd column queen location.

4 Queens

1	2	3	4
Q			

Q			
	Q		

1			
			2
	3		

1			
			2
	3		
x	x	x	x

BACK

	1		

BACK

	1		
			2
3			

1	1		
3			2
		4	

8 Queens,

8, 2, 7, 1, 3, 5, 4, 6
2, 7, 1, 3, 5, 4, 6

	1	2	3	4	5	6	7	8
1						Q ₆		
2				Q ₄				
3							Q ₇	
4	Q ₁							
5								Q ₈
6		Q ₂						
7					Q ₅			
8			Q ₃					

Job Sequencing (Greedy Techn.)

(Optimal answer)

* 1 unit of time.

	J_1	J_2	J_3	J_4
Profit:	50	15	10	25
Deadline:	2	1	2	1

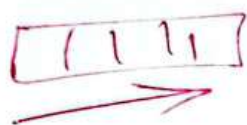
For making software
↳ profit
↳ deadline,

* Uniprocessor (1 person working)

* No preemption (H H H)

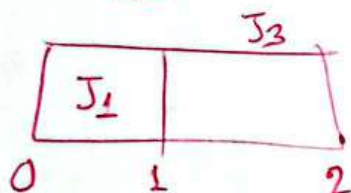
Algorithm:

1) Arrange all jobs in decreasing order of profit. [n log n]



2) For each job (m.p) do 'linear search' to find particular slot in array of size (n), where $n = \max$ deadline

Gantt chart



50 profit,

1st max

J_4

10

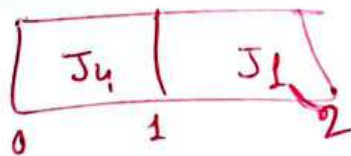
$$50 + 10 = 60$$

(2 deadline)

as it should be completed in 1st month,

Time: $n \times m = mn$
if equal then $n \times m$

etc
or highest
1st beg (p.s)



IPS write 1st max to near to deadline,

$$25 + 50 = 75$$

* optimal value.

(J1)

(Base)
Radix Sort,

non-comparison based algo,

3 digit
904
2 digit
046
1 digit
005
074
062
001

append 0 as to highest digits,

focus on least significant bit,
(unit)

* don't change relative position,
follows (stable sort technique)

↓ ↓ ↓ ↓ ↓
001 - 062 - 904 - 074 - 005,
 1st
 it was
 046

⇒ then come to tens place,

↓ ↓ ↓ ↓ ↓
001 - 904 - 005 - 046 - 062
 074
MSB
✓ 001 - 005 - 046 - 062 - 074 - 904
sorted

Selection Sort

the elem which is min ~ it will bring
that min elem to the START (with 1 Turn)

2nd min to 2nd front (2nd Turn)
↓ ↓

Selection Sort;

for $i \leftarrow 1$ to $n-1$ do

min $j \leftarrow 1$;

min $x \leftarrow A[i]$

min = 40

for $j \leftarrow i+1$ to n do

If $A[j] < \text{min } x$ then

min $j \leftarrow j$

min $x \leftarrow A[j]$

$A[\text{min } j] \leftarrow A[i]$

$A[i] \leftarrow \text{min } x$

$i=1$
40

20

60

10

50

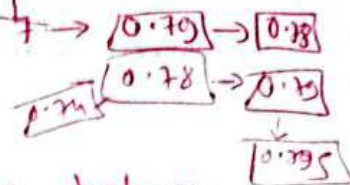
30

j

Bucket Sort

(non-comparison Based)

intensity $\Rightarrow 0.75, 0.78, 0.795$
 $0(n) \times n = 0(n^2)$



straight/ - Works on floating-point numbers between range 0.0 to 1.0,

- Inputs should be uniformly and independently distributed across $[0,1]$ to get a running time of $O(n)$,

Dot/Radix

0.75, 0.13, 0.64, 0.39, 0.20, 0.89, 0.53, 0.42, 0.06, 0.94

0	0.06
1	0.13
2	0.20
3	0.39
4	0.42
5	0.53
6	0.64
7	0.75
8	0.89
9	0.94

1 bucket $\Rightarrow$ bucket/array

n A[i]
 10×0.75

= lower value

Floor

[7.5]

7

[1.3]

1

Best

n - traverse times
 order of 1 time, $O(1)$

$O(n)$
 given

Bucket Sort(A)

- ① let $B[0..n-1]$ be a new array.
 - ② $n = A$ length
 - ③ for $i = 0$ to $n-1$
 - ④ make $B[i]$ an empty list
 - ⑤ for $i = 1$ to n
 - ⑥ insert $A[i]$ into list $B[\lfloor n A[i] \rfloor]$
 - ⑦ for $i = 0$ to $n-1$
 - ⑧ sort list $B[i]$ with insertion sort,
 - ⑨ concatenate the lists together
- pick up $B[0], B[1], \dots, B[n-1]$ (in order)
 ascending order

Count sort (non-comparison based)
 ↳ based on stable sort

↳ relative order
 of values
 in original array should
 not be disturbed,

Input size - 'n'

Range - 'k'

range
 eg: (1-5) →

2 1 3 1 2

↳ Take

extra
 memory

2	1	Traverse ↓
2	2	
1	3	
1	4	
0	5	

size $O(n+k)$

T.C =

S.C = extra space

$O(k)$

1 1 2 2 2 3 4
 Sort ✓

↳ Works in (Linear Time)

prob: Range (predefined),

2 20000 3 6 7
 ↳ by value,



Port City International University
Final Term Examination, Fall-2023 Trimester

Course Code: CSE 227
 Department: CSE
 Time: 2.00 hours

Course Title: Algorithms
 Batch: CSE-27-A (Day)
 Full Marks: 40

[N.B.: Answer any Five (5) Questions. Figures in the right margin indicate full marks.]

1. Consider the Fig 1. for solving 1(b):

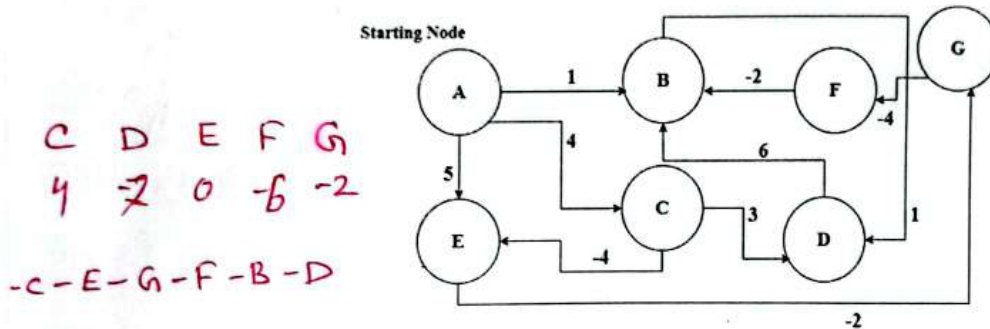


Fig 1. Directed Graphical Representation.

- a) Derive the time complexity of Bellman-Ford algorithm. $O(V^3)$; $O(VE)$ 2 CLO3
 b) Find out the shortest path using the Bellman-Ford algorithm from the above graph. 6 CLO3

2. In computer science, Prim's algorithm (also known as Jarnik's algorithm) is a greedy algorithm that finds a minimum spanning tree for a weighted undirected graph.

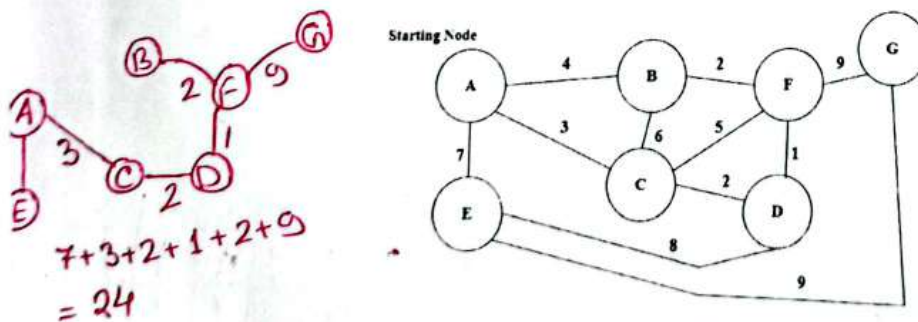
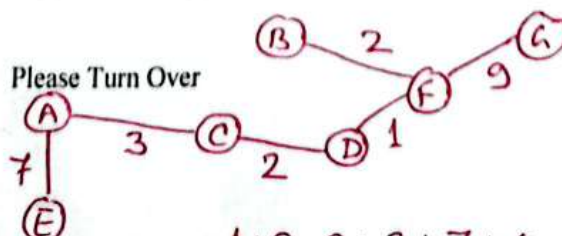


Fig 2. Undirected Graphical Representation.

- a) Analyze the time complexity of Prim's algorithm. S.C: $O(E+V)$ T.C: $O(V^2)$; $O((E+V)\log V)$ 2 CLO3
 b) Design a graph using Prim's algorithm considering Fig.2 as reference. 6 CLO3

3. a) "Prim's algorithm is faster than Kruskal's algorithm."- Justify your answer. 2 CLO3
 b) Modify a graph using Kruskal's algorithm in light of Fig. 2. 6 CLO3

A-B = 4 X D-E = 8 X
 A-C = 3 ✓ F-G = 9 ✓
 A-E = 7 ✓ E-G = 9 ✓
 B-C = 6 X
 B-F = 2 ✓
 C-F = 5 X
 C-D = 2 ✓
 D-F = 1 ✓

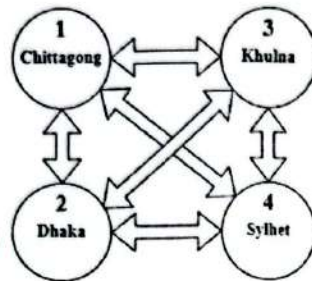
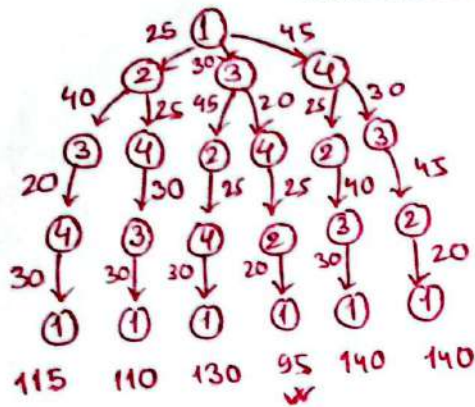


$1+2+2+3+7+9 = 24$

ALGO

$$5+5+8+5+$$

4. To solve the TSP using the Brute-Force approach, you must calculate the total number of routes and then draw and list all the possible routes.



	1	2	3	4
1	0	25	30	45
2	20	0	40	25
3	30	45	0	20
4	30	25	30	0

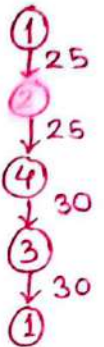


Fig 3. Travelling Salesman Problem (TSP).

$$25+25+30+$$

Demonstrate the above figure using travelling salesman problem to find the 8 CLO minimum cost.

5. Consider the Fig 4. for solving the following questions:

	0	1	2	3	4	5	6
0	0	0	0	1	1	0	1
1	0	0	1	1	0	0	0
2	0	1	0	0	1	0	0
3	0	1	0	0	0	0	1
4	0	1	0	1	0	0	1
5	0	0	0	0	1	0	1
6	1	0	0	0	1	1	0

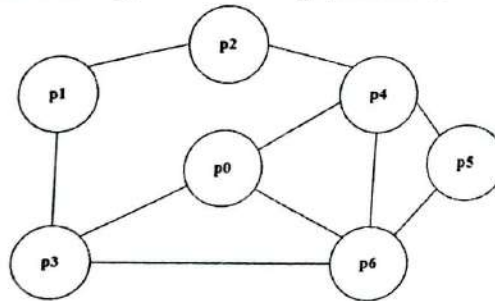


Fig 4. Graph (undirected).

- Isolate the graph with the help of adjacency matrix.
- Describe adjacency list in respect of the above figure.

$$4 \text{ CLO} \text{ (B)}$$

6. Answer the following question using the given data:

item: 1 2 3 4 5

weight: 4 3 2 1 3

reward: 6 7 8 9 11

max weight: 5

item, $n = 5$

Sketch the maximum profit using 0/1 knapsack problem.

	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	0	0	0	6	6
2	0	0	0	7	7	7
3	0	0	8	8	8	15
4	0	9	9	17	17	17
5	0	9	9	17	20	20

$$8 \text{ CLO}$$

- Distinguish between greedy algorithm and dynamic programming.
- Trace a graph with six vertices to explain Dijkstra's SSSP algorithm.

$$4 \text{ CLO}$$

$$4 \text{ CLO}$$

Port City International University
Final Term Examination, Fall-2023 Trimester

Course Code: CSE 227
 Department: CSE
 Time: 2.00 hours

Course Title: Algorithms
 Batch: CSE-28-A (Evening)
 Full Marks: 40

[N.B.: Answer any Five (5) Questions. Figures in the right margin indicate full marks.]

1. Consider the Fig 1, for solving 1(b):

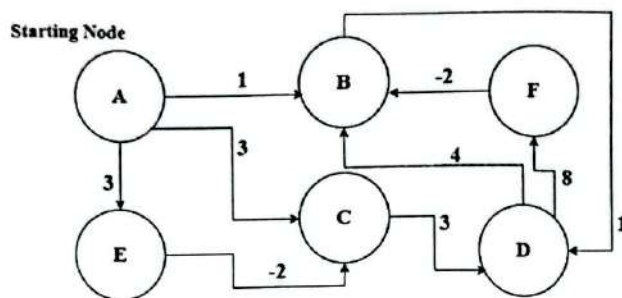


Fig 1. Directed Graphical Representation.

- a) Derive the time complexity of Bellman-Ford algorithm. $O(V \cdot |E|)$ 2 CLO3
 b) Construct the above graph to find the shortest path using Bellman-Ford algorithm. $0 + 1 + 1 + 2 + 3 + 9 = 16$ 6 CLO3

2. In computer science, Prim's algorithm (also known as Jarnik's algorithm) is a greedy algorithm that finds a minimum spanning tree for a weighted undirected graph.

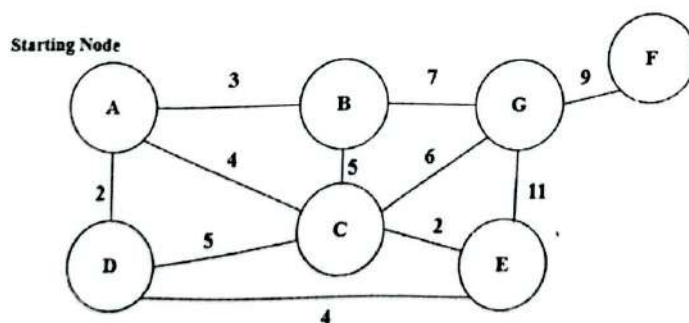
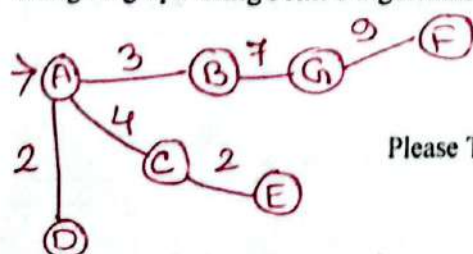
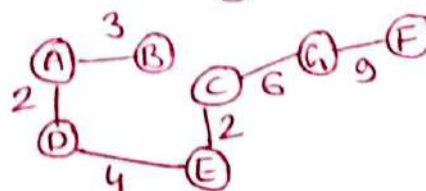


Fig 2. Undirected Graphical Representation.

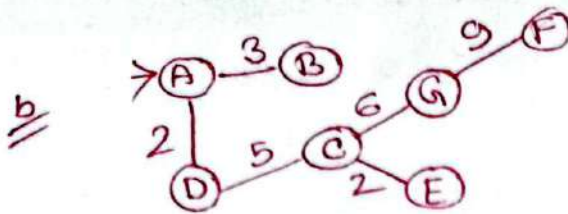
- a) Analyze the time complexity of Prim's algorithm. Adjacency Matrix: $O(V^2)$ Binary heap: $O(V \log V + E \log V)$ 2 CLO3
 b) Design a graph using Prim's algorithm considering Fig.2 as reference. 6 CLO3



Total cost: $3 + 2 + 2 + 5 = 12$



26

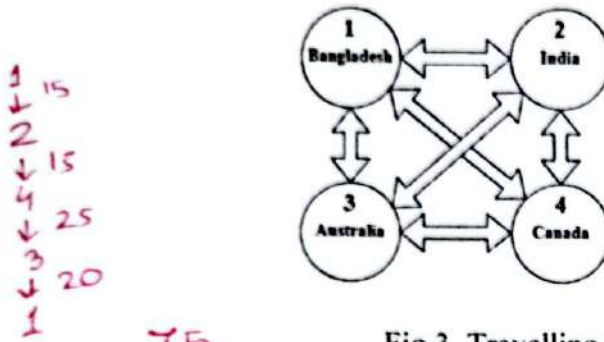


Total cost: $3+2+5+2+6+9$
 $= 27$

Top Down Approach
 more Efficient

3. a) "Prim's algorithm is faster than Kruskal's algorithm." - Justify your answer. 2 CL
 b) Modify a graph using Kruskal's algorithm in light of Fig. 2. 6 CL

4. To solve the TSP using the Brute-Force approach, you must calculate the total number of routes and then draw and list all the possible routes.



	1	2	3	4
1	0	15	20	25
2	10	0	30	15
3	20	35	0	10
4	20	15	25	0

Fig 3. Travelling Salesman Problem (TSP).

Demonstrate the above figure using travelling salesman problem to find the 8 CL minimum cost.

5. Consider the Fig 4. for solving the following questions:

	q_0	q_1	q_2	q_3	q_4	q_5
q_0	0	1	1	1	0	0
q_1	1	0	0	1	1	1
q_2	1	0	0	0	1	0
q_3	1	1	0	0	0	1
q_4	0	1	1	0	0	1
q_5	0	1	0	1	1	0

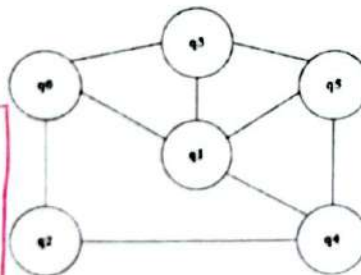


Fig 4. Graph (undirected).

- a) Isolate the graph with the help of adjacency matrix. 4 CL
 b) Describe adjacency list in respect of the above figure. 4 CL

6. Answer the following question using the given data:

item: 1 2 3 4 5

weight: 3 1 3 4 2

reward: 6 8 5 9 7

max weight: 4

item, $n = 5$

Sketch the maximum profit using 0/1 knapsack problem.

	0	1	2	3	4
0	0	0	0	0	0
1	0	0	0	0	0
2	0	8	8	8	14
3	0	8	8	8	14
4	0	8	8	8	14
5	0	8	8	15	18

7. a) Distinguish between greedy algorithm and dynamic programming. 2 CL
 b) Trace a graph with eight vertices to explain Dijkstra's SSSP algorithm. 6 CL

Top down
 Faster
 more Efficient
 eg. Fractional Knapsack.

Bottom-up
 Slower
 Less efficient
 eg. 0/1 Knapsack.